Компиляторы

Сравнение прямой свёртки и алгоритма Винограда

Галицын Матвей Дмитриевич
Группа: Б01-411

11 апреля 2026 г.

Содержание

1 Описание классической свёртки **3**

2 Описание алгоритма Винограда **3**

3 Экспериментальные результаты **4**

3.1 CPU 4

3.2 GPU 5

4 Вывод **6**

1 Описание классической свёртки

Наивный алгоритм свёртки работает следующим образом. Для каждого выходного пикселя выбирается входной тайл, размер которого совпадает с размером ядра. Для каждого канала выполняется поэлементное умножение значений тайла и соответствующего слоя ядра. Далее результаты суммируются сначала по каналам, а затем по всем элементам тайла.

Таким образом, значение одного выходного пикселя вычисляется как сумма произведений элементов входа и ядра. После этого окно сдвигается на один пиксель (stride = 1), и процесс повторяется для всех позиций и всех изображений батча.

2 Описание алгоритма Винограда

В работе реализован алгоритм Винограда $F(2,3)$, который вычисляет блок выхода 2×2 с использованием входного тайла 4×4 и ядра 3×3.

Алгоритм состоит из следующих этапов:

- Преобразование ядра (выполняется один раз):

$$transformedKernel = G \cdot kernel \cdot G^T$$

- Преобразование входного тайла:

$$transformedInput = B^T \cdot tile \cdot B$$

- Поэлементное умножение:

$$M = transformedKernel \odot transformedInput$$

- Обратное преобразование:

$$Y = A^T \cdot M \cdot A$$

Главное преимущество метода — уменьшение количества умножений: вместо 36 операций (для 2×2 блока) требуется всего 16.

Также важно отметить, что тайл в алгоритме Винограда сдвигается на 2 пикселя, а не на 1, как в наивной реализации.

3 Экспериментальные результаты

3.1 CPU

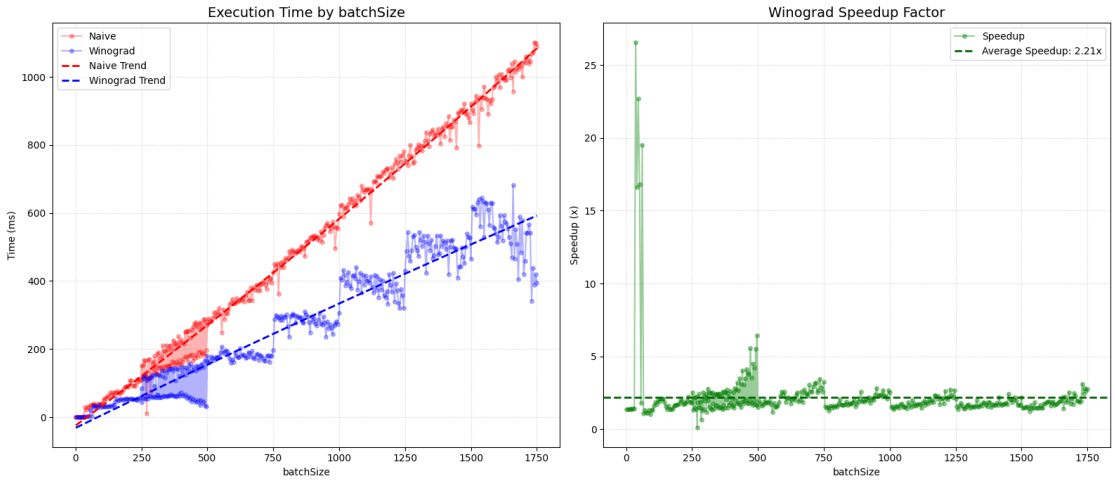

Рис. 1: Зависимость времени алгоритма от размера батча

Алгоритм Винограда демонстрирует среднее ускорение в 2.21×, что является отличным результатом. Наблюдаются выбросы на границах диапазонов, что связано с особенностями сбора данных (разные файлы, холодный кэш).

После значения batchSize ≈ 750 наблюдается плато: ускорение стабилизируется около 2×. Это означает, что оба алгоритма начинают расти линейно по времени, и выигрыш становится постоянным.

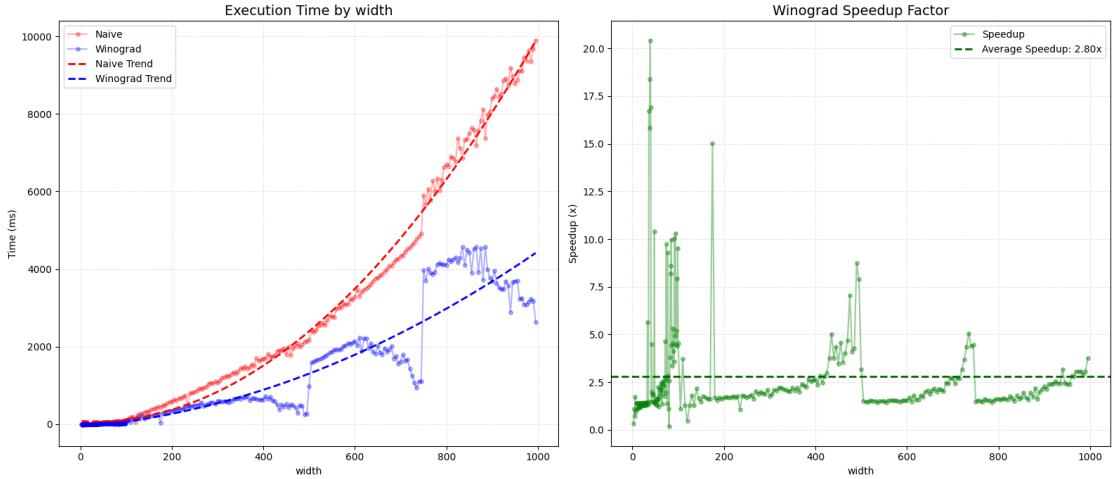

Рис. 2: Зависимость времени алгоритма от размера тензора

Сложность свёртки растёт как $O(W^2 \cdot kH \cdot kW \cdot C)$, поэтому зависимость имеет квадратичный характер.

При увеличении размера тензора внутренний цикл по тайлам становится доминирующим. За счёт этого разница между алгоритмами накапливается, и ускорение увеличивается.

Так как в реализации данные хранятся последовательно, алгоритм Винограда за одну операцию обрабатывает 4 элемента, а наивный — 3. Это даёт дополнительный вы-

игрыш, который теоретически оценивается как 1.3–1.7 раза. В результате наблюдается средний speedup около 2.8×.

3.2 GPU

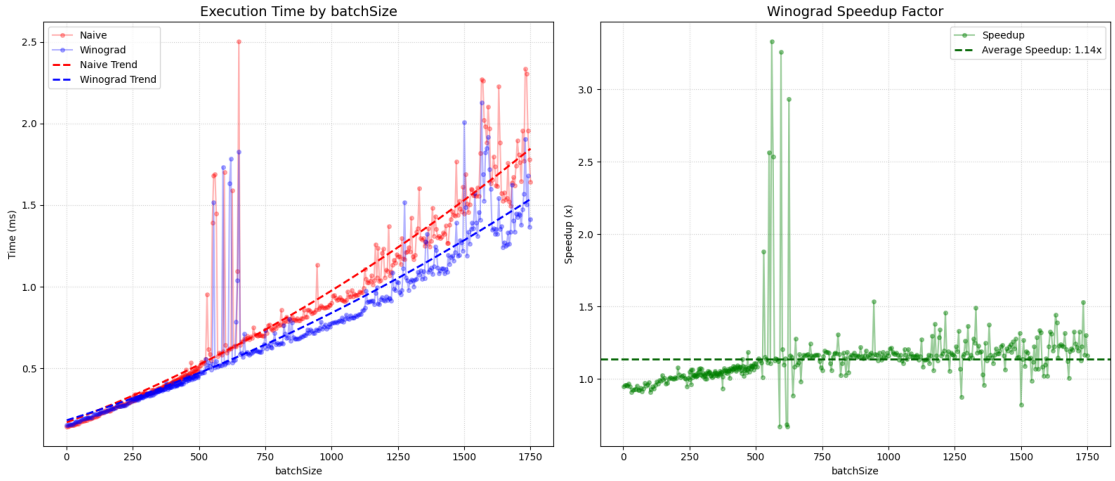

Рис. 3: Зависимость времени алгоритма от размера батча

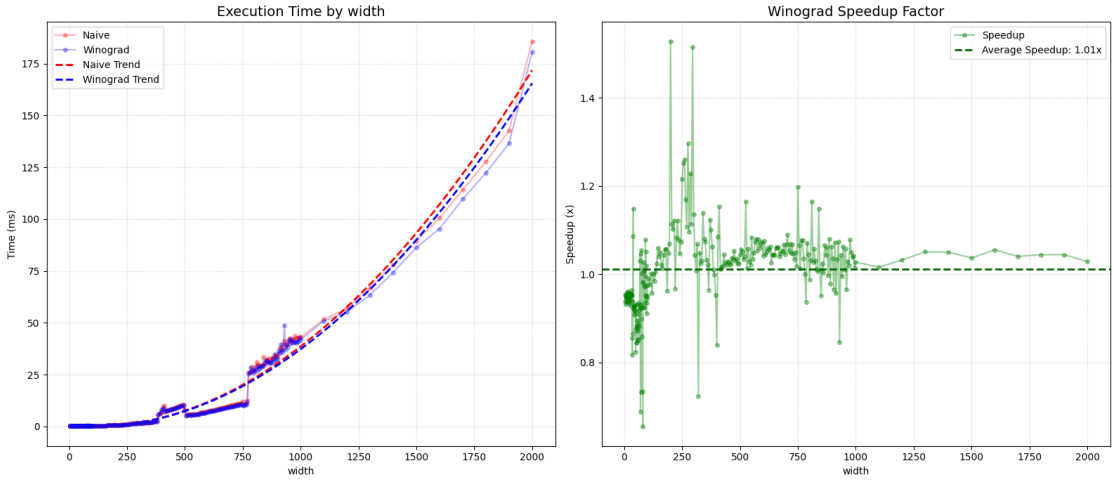

Рис. 4: Зависимость времени алгоритма от размера тензора

Оба алгоритма одинаково хорошо параллелятся по (x, y, batch) в NDRange. GPU загружен в обоих случаях одинаково. А разница между ними только в том сколько умножений происходит внутри одного work-item — и это сокращение просто теряется на фоне того что GPU и так гоняет тысячи work-items параллельно по батчу.

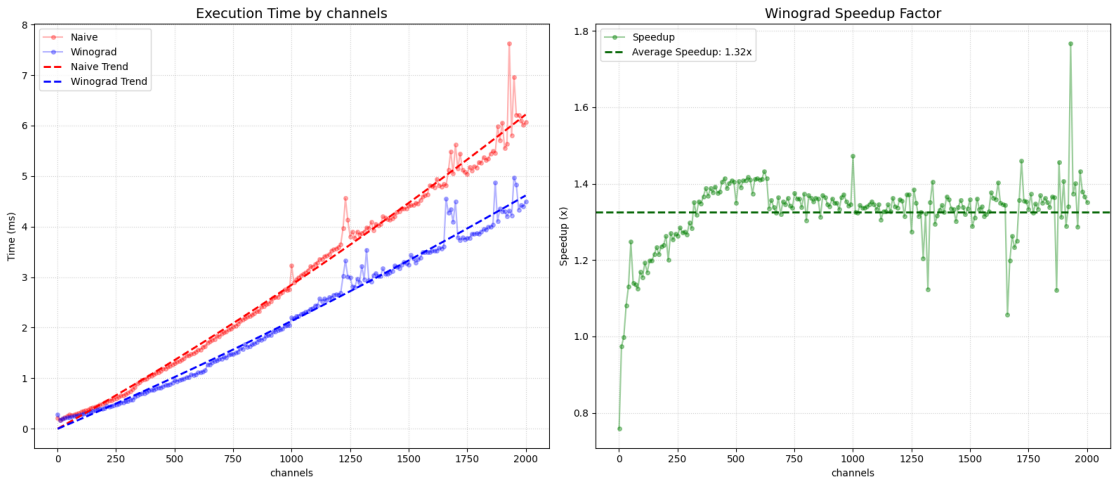

Рис. 5: GPU: зависимость от числа каналов

При увеличении числа каналов преимущество алгоритма Винограда становится заметнее, так как в данной реализации параллелизация по каналам отсутствует, и выигрыш от уменьшения числа операций проявляется сильнее.

Итак, максимальное ускорение достигается при больших размерах тензора. В этом случае число обрабатываемых тайлов велико, и выигрыш от уменьшения количества операций на каждом тайле суммируется. Экспериментально наблюдается ускорение до $\approx 2.8\times$.

Размер батча влияет на эффективность следующим образом: при увеличении N ускорение сначала растёт, а затем выходит на плато (около $2\times$). Это связано с тем, что оба алгоритма начинают масштабироваться линейно, и относительное преимущество становится постоянным.

4 Вывод

Алгоритм Винограда показывает значительное ускорение по сравнению с наивной реализацией за счёт уменьшения количества умножений. На CPU выигрыш особенно заметен при больших размерах входных данных. На GPU преимущество уменьшается из-за высокой степени параллелизма, однако при определённых условиях (например, рост числа каналов) оно также проявляется.